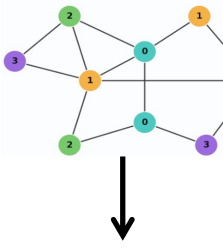


Training

Input graph G



C condition

C is a global graph representation.

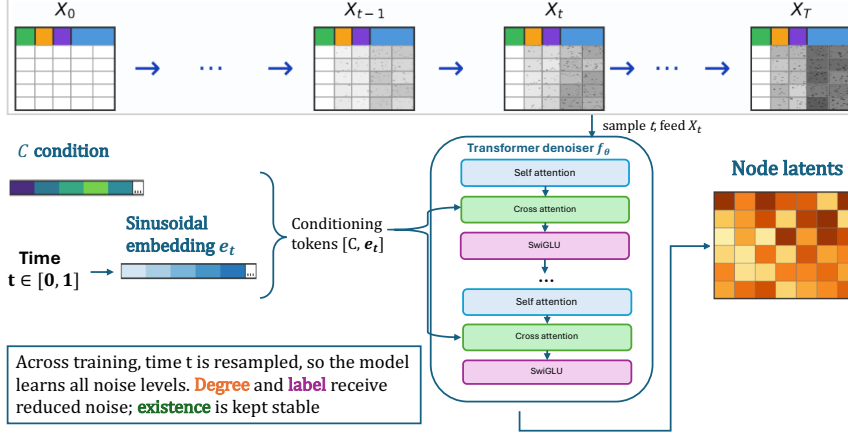
Node features X_0

node exists	degree	label	features
1	2	3	0.12 -0.43 ...
1	1	1	-0.35 0.21 ...
1	3	0	0.28 0.54 ...
...	...	...	...
1	1	0	0.44 -0.12 ...

X_0 stores one row per possible node. Columns encode node existence, degree, node label, and continuous node features.

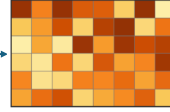
The input graph is decomposed into local node information X_0 and global graph context C .

Noise schedule over node features



Across training, time t is resampled, so the model learns all noise levels. **Degree** and **label** receive reduced noise; **existence** is kept stable

Node latents Z

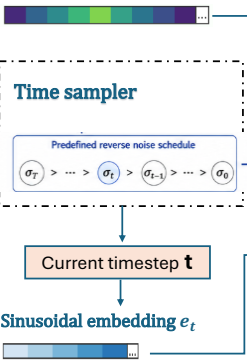


$$L = L_{noise} + \lambda_{exist} L_{exist} + \lambda_{node_lbl} L_{node_lbl} + \lambda_{deg} L_{deg} + \lambda_{edge} L_{edge} + \lambda_{edge_lbl} L_{edge_lbl} + \lambda_{dist} L_{dist} + \lambda_{cons} L_{cons}$$

Head/term	Supervision Type	Loss symbol	Training Objective
Noise	Continuous node wise	L_{noise}	Denoise node features
Node existence	Binary node wise	L_{exist}	Predict valid nodes
Node label	Categorical node wise	L_{node_lbl}	Predict node type/label
Degree	Categorical node-wise	L_{deg}	Predict node degree
Edge existence	Pairwise binary	L_{edge}	Predict edges between node pairs
Edge label	Categorical pairwise	L_{edge_lbl}	Predict edge type/label
Distance head	Categorical pairwise	L_{dist}	Predict the distance between node pairs
Consistency term	Regulariser/not a head	L_{cons}	Keep absent nodes empty

Sampling and decoding

Choose condition C



Reverse denoising loop

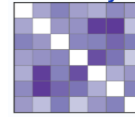


Reverse steps reduce the noise for the continuous features. The heads give **existence**, **degree** and **label** predictions

Transformer denoiser f_θ (reused at every step); $f_\theta(X_t, C, e_t) \rightarrow X_{t-1}$

Outputs at time $t=0$

Edge probability matrix P (from edge head)



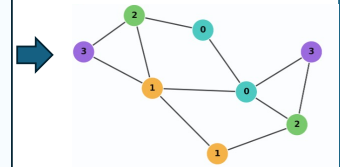
Generated node features $\hat{X}_0$

node exists	degree	label	features
1	2	1	0.31 -0.18 ...
1	3	0	-0.22 0.47 ...
1	1	2	0.08 0.63 ...
...	...	...	...
1	2	3	-0.41 0.12 ...

Constrained graph decoder

1. Threshold node-existence scores
2. Mask absent/padded nodes
3. Symmetrize edge-head probability matrix
4. Solve an ILP with degree constraints/slack to obtain a valid undirected adjacency
5. Optionally enforce connectivity
6. Decode node labels and optional edge labels

Decoded graph $\hat{G}$



Notation: G graph | C graph level-condition vector | X_0 clean node features | X_t, X_T noised node features at time t, T | $\hat{X}_0$ clean generated node features | P edge existence head probability matrix | λ_* loss weights | Z node latents produced by the transformer | e_t sinusoidal time embedding t | σ_t scheduled noise scale used by the sampler/noise schedule | L overall loss